pyQGIS를 이용한 Map Image to Unity Terrain 변환 시스템 제작

김현민

Creating a Map Image to Unity Terrain conversion system using pyQGIS

Kim HyeonMin

초록(Abstract)

최근 주요 IT 기업의 신규 VR 디바이스 출시로 VR 게임에 관한 관심이 급증하고 있다. VR 기술의 몰입형 가상 현실을 구현하기 위해 현실 공간 데이터를 게임 공간으로 효과적으로 변환하는 기술이 필요할 것이라 생각한다. 이에 본 연구에서는 이미지 데이터를 기반으로 tif 형식의 지리 파일을 제작하고 이를 Unity의 Terrain으로 변환하는 자동화 시스템을 제안한다. 먼저 MiDaS 모델을 이용해 이미지의 깊이 파악을 통해 지형 정보를 추출하고, 이를 기반으로 높이 배열을 생성한다. 이후 pyQGIS를 이용해 생성된 데이터를 기반으로 맵 변환을 통해 .tif 형식으로 출력한다. 마지막으로 Unity 환경에서 Terrain 오브젝트로 변환하여 elevation 데이터와 텍스처를 적용한다. 이 시스템을 사용하면 게임산업에서 맵을 디자인 하는 데에 필요한 반복 작업을 대체함으로써 더욱 효율적인 콘텐츠 제작이 가능할 것으로 기대된다.

주제어: VR, MiDaS, Terrain, pyQGIS

Ⅰ. 서론

가상현실(VR) 기술은 몰입형 경험을 제공하며 게임, 교육, 훈련 등 다양한 산업에서 활용 가능성을 넓혀가고 있다. 특히 주요 IT 기업들이 신규 VR 기기를 잇달아 출시하면서 VR 콘텐츠의 수요가 증가하고 있다. 이러한 변화 속에서 현실 세계를 기반으로 한 VR 콘텐츠는 사용자에게 더 현실적이고 생동감 있는 경험을 제공할 수 있어 큰 관심을 받고 있다. 하지만 현실 공간을 가상 공간의 맵으로 변환하는 과정은 복잡하고 많은 시간과 비용 및 높은 수준의 기술력이 요구된다. 이러한 VR 콘텐츠 제작의 효율성을 높이고 비용을 절감하기 위해서는 작업 과정을 자동화하고 단순화할 수 있는 시스템이 필요하다. 본 연구에서는 MiDaS 모델과 pyQGIS를 이용해 2D 이미지 데이터를 기반으로 현실의 지형 정보를 분석하고, 이를 활용해 Unity의 Terrain 시스템으로 변환하는 자동화된 시스템을 제안한다. 이 시스템을 사용하면 현실 지형 데이터를 가상 공간 맵으로 신속하고 효율적으로 변환하는 시스템을 개발함으로써 VR 콘텐츠 제작 과정의 비용과 시간을 대폭 절감할 수 있을 것으로 기대된다.

Ⅱ. 이론적 배경

1. MiDas

1) 깊이 추정(Depth Estimation)

깊이 추정은 2D 이미지에서 각 픽셀이 카메라와 얼마나 떨어져 있는지를 추정하는 작업으로, 컴퓨터 비전에서 중요한 연구 분야이다. 깊이 정보는 증강현실(AR), 가상현실(VR) 등 다양한 응용 분야에

서 활용된다. 기존에는 스테레오 카메라나 라이다(LiDAR) 센서와 같은 하드웨어 기반의 기술이 사용되었지만, 앞으로는 단일 이미지로부터 깊이를 추정하는 기술이 좀 더 필요할 것이다. AI 기술이 발전하면서 정확도가 높아졌으며 이러한 방법은 기존의 방법보다 효율적이며, 다양한 환경에서 활용이 가능할 것이라 여겨지기 때문이다.

2) MiDaS

MiDaS 모델은 단일 이미지로부터 상대적 깊이를 추정하기 위한 딥러닝 기반의 최첨단 모델이다. 이 모델은 다양한 입력에 대해 높은 품질을 보장하기 위해 다목적 최적 설계를 사용하여 10개의 개별 데이터세트에 대해 훈련되었으며 이 덕분에 단일 이미지에서 깊이 정보를 추출하는 데 다양한 조명 조건, 환경, 그리고 복잡한 장면에 상관없이 균등하게 탁월한 성능을 보여준다.

3) 응용 분야

MiDaS 모델은 다양한 분야에 적용될 수 있다. 예를 들어 자율 주행 분야에서는 단일 카메라를 통해 도로의 깊이 정보를 추출하여 주행 환경을 파악하는 데 활용될 수 있다. 증강현실(AR)에서는 깊이 정보를 기반으로 객체를 적절히 배치하고 상호작용을 구현할 수 있으며, 가상현실(VR)에서는 현실 세계를 3D로 변환하여 사용자에게 더욱 몰입감 있는 환경을 제공할 수 있다. 또한, 2D 이미지를 3D 모델로 변환하는 작업에도 유용하게 사용될 수 있다.

2. pyQGIS

1) GIS(Geographic Information System)

GIS는 지리 데이터를 수집, 저장, 분석, 시각화하는 데 사용되는 시스템으로, 지형 분석, 도시 계획, 환경 관리, 재난 관리 등 다양한 분야에서 활용된다. GIS 소프트웨어는 사용자가 위치 기반 데이터를 처리하고 시각적으로 이해할 수 있도록 지원한다. 이 중 QGIS는 오픈 소스 기반 GIS 소프트웨어로, 강력한 기능과 확장성을 제공하여 실무에서 널리 사용되고 있다.

2) pyQGIS

pyQGIS는 QGIS의 기능을 Python을 이용해 사용할 수 있도록 하는 라이브러리로, 지리 데이터를 프로그래밍 방식으로 처리, 분석, 변환하는 데 유용하다. pyQGIS는 QGIS 코어와 긴밀히 통합되어 있어, GUI(Graphical User Interface)를 사용하지 않고도 대규모 데이터를 처리하거나 복잡한 지리 분석 작업을 수행할 수 있다.

3) 응용 분야

pyQGIS는 다양한 분야에서 활용될 수 있으며, 특히 대규모 지리 데이터를 분석하거나 반복 작업을 자동화해야 하는 경우 유용하다. 주요 응용 분야로는 도시에서 공간 데이터를 활용하여 인프라 설계 및 자원 배치를 최적화하는 데 사용되거나, 환경 분석에서 생태계 변화, 오염 추적, 자연재해 분석을 하는 데에 사용될 수 있다. 또한, 지리 데이터를 기반으로 가상 환경을 만들고, 이를 게임이나 가상현실에 적용하는 데 사용될 수도 있다.

Ⅲ. 연구 내용 및 방법

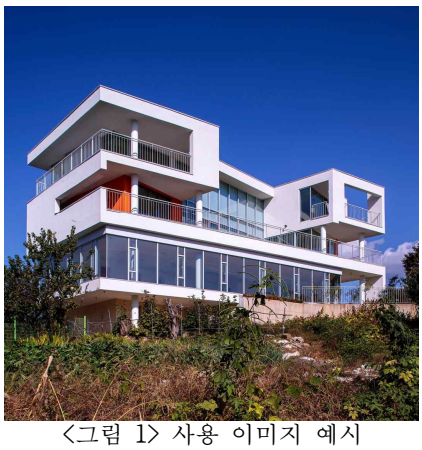
1. 연구의 목표

본 연구는 MiDaS 모델을 활용하여 정면 사진으로부터 깊이 맵을 생성하고, 이를 Unity의 Terrain 지형 모델로 변환하는 시스템을 설계 및 구현하는 데에 목적이 있다. 이를 통해 2D 이미지를 기반으로 한 3D 지형 생성의 가능성을 확인하고자 한다.

2. 연구 계획 및 방법

1) MiDaS 모델을 이용한 이미지 깊이 맵 제작

본 연구에서는 풍경 사진이나 그림과 같은 데이터를 활용하여 깊이 맵을 생성하는 방법을 개발하고자 한다. 이를 위해 인터넷에서 수집한 여러 풍경 이미지 중 연구의 특성상 정면에서 촬영된 사진을 중심으로 데이터를 선정한다. 이미지의 화질이 낮거나 비율이 1:1에 크게 벗어난 경우, 혹은 촬영 각도가 정면에서 크게 벗어난 이미지는 분석에서 제외한다.



<그림 1> 사용 이미지 예시

이미지의 깊이 정보를 추정하기 위해 MiDaS 모델을 채택한다. MiDaS는 Transformer 기반의 아키텍처를 활용하여 깊이 추정 작업에 탁월한 성능을 보이는 모델로, 파이토치를 통해 사전 학습된 모델을 가져와 사용한다. 그러나, MiDaS 모델은 대형 이미지에 대해 실시간 추론이 어려운 단점이 있으므로, 이미지 크기를 적절히 조절해 사용한다. 깊이 맵 생성 과정에서, 깊이 값을 0에서 100 사이로 보정하여 정성적인 구현이 가능하도록 데이터를 정규화한다. 보정된 깊이 데이터를 matplotlib 라이브러리를 이용하여 시각화하고, 원본 이미지와 깊이 맵을 함께 표현함으로써 깊이에 따라 색상이 변하는 그래프 형태로 결과를 확인한다.

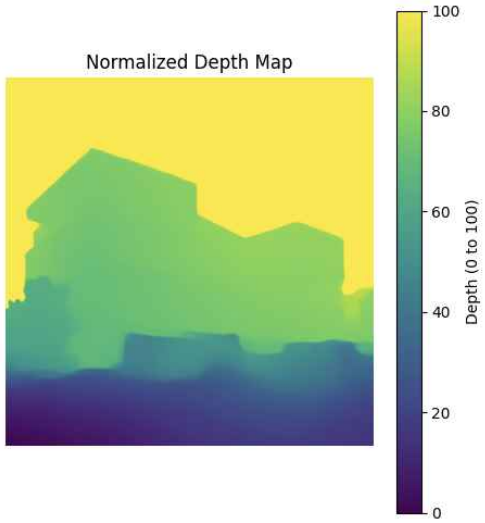
2) 깊이 맵을 높이 맵 및 Unity Terrain으로의 변환

MiDaS 모델로부터 생성된 깊이 맵은 2차원 배열 형태로, 이미지의 정면에서 본 깊이를 표현한다. 이를 Unity의 Terrain 지형 데이터로 변환하기 위해, 깊이 맵을 위에서 높이 맵으로 변환하는 과정이 필요하다. 깊이 데이터를 위에서 본 높이 맵으로 변환하기 위해 데이터가 각 위치의 지형의 높이 값이 되도록 변환한다. 변환된 높이 데이터는 pyQGIS의 GDAL 라이브러리를 이용해 .tif 형식의 파일로 저장되었으며, 이는 지형의 고도 데이터를 표현하는 데 적합한 포맷으로 Unity의 Terrain 시스템에서 활용할 수 있다. 이후 .tif 파일을 텍스처 이미지를 이용해 Unity에서 Terrain 오브젝트를 생성한 뒤 변환된 Texture2D 데이터를 적용하고 정보를 Terrain에 매핑해 3D 지형을 생성한다.

Ⅳ. 연구 결과

1) MiDaS 모델을 이용한 이미지 깊이 맵 제작

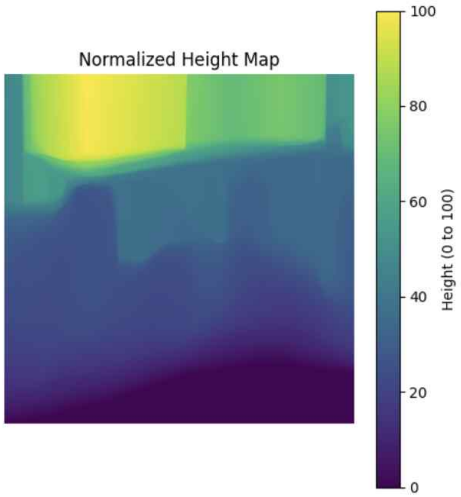
본 연구에서는 풍경 사진이나 그림과 같은 비정형 데이터를 활용하여 깊이 맵을 생성하는 시스템을 개발했다. 사진은 인터넷에서 수집한 고화질 이미지 중 연구의 특성상 정면에서 촬영된 사진을 중심으로 데이터를 선정했다. 이미지의 화질이 낮거나 비율이 1:1에 크게 벗어난 경우, 혹은 촬영 각도가 정면에서 크게 벗어난 이미지는 분석에서 제외했다. 이미지의 깊이 정보를 추정하기 위해 파이토치의 사전 학습된 MiDaS 모델을 가져와 사용했다. 깊이 맵 생성 과정에서, 깊이 값을 0에서 100 사이로 보정하여 정성적인 구현이 가능하도록 데이터를 정규화했으며 보정된 깊이 데이터를 matplotlib 라이브러리를 이용하여 시각화했다. 원본 이미지와 깊이 맵을 함께 표현함으로써 깊이에 따라 색상이 변하는 그래프 형태로 결과를 확인했다.



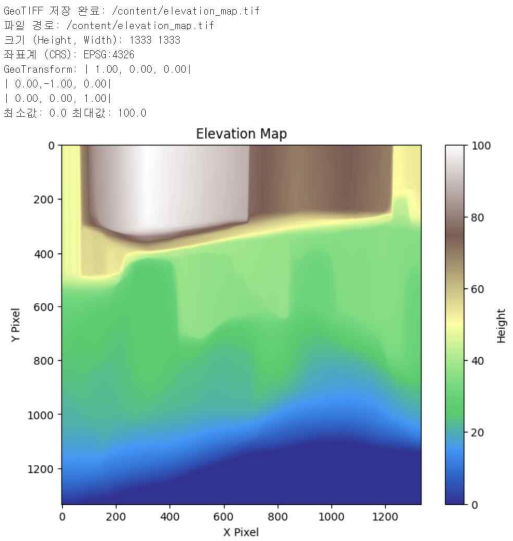
<그림 2> 정규화된 깊이 맵

2) MiDaS 모델을 이용한 이미지 깊이 맵 제작

MiDaS 모델로 생성된 깊이 맵은 Numpy 배열 형태로, 정면에서 본 깊이를 나타낸다. 이 깊이 맵을 Unity의 Terrain 지형 데이터로 변환하기 위해 깊이 맵을 0에서 100 사이의 값으로 표준화하고 이 과정에서 영향이 적은 특정 값 이하는 절삭하거나 보정하여 데이터의 품질을 향상했다. 이후, 깊이 맵을 높이 맵으로 변환하는 과정을 진행했다. 원본 사진이 완전히 정면에서 촬영되었다고 가정하고, 배열의 행(높이)과 열(너비)을 순회하면서 각 셀의 값을 활용하여 탑뷰에서 영향을 받는 부분 열의 값을 조정했으며 이때 멀리 있는 물체는 더 적은 영향을 주고, 가까운 물체는 더 큰 영향을 주도록 설정하여 위에서 내려다본 시점의 높이 정보를 포함하는 2D 높이 맵을 생성했다. 변환된 높이 맵은 matplotlib를 이용해 원본 사진과 함께 시각화했고, 색상을 활용하여 높이 정보를 나타내는 높이 맵과 3D 맵으로 표현했다. 완성된 높이 맵은 .tif 형식으로 저장하여 Unity Terrain 시스템에서 활용할 수 있도록 준비했다.



〈그림 3〉 정규화된 높이 맵



〈그림 4〉 .tif 파일의 Elevation Map

Ⅴ. 결론 및 제언

MiDaS 모델을 이용해 깊이 추정을 통해 생성된 맵은 탑 뷰의 특성을 활용한 변환을 통해 정면 이미지 일부를 높이 맵으로 변환할 수 있음을 확인했다. 또한 이렇게 만들어진 높이 2차원 배열을 .tif 형식의 파일로 저장하여 게임 엔진에서 지형 맵으로 사용할 수 있었다. 그러나 MiDaS 모델은 빠른 작동 속도와 정면 이미지에서의 높은 정확성을 보여주었으나, 이미지를 측면에서 촬영했을 때는 성능이 크게 저하되는 문제가 있었다. 또한, 정면 이미지만을 활용하여 높이 맵을 생성하는 과정에서 배열의 순회를 통해 값을 변환하므로 시간이 오래 걸렸으며, 지형이 복잡한 형태일 경우 정확한 모양을 표현하는 데 한계가 있었다. 이러한 문제를 해결하기 위해 향후 연구에서는 행렬 및 벡터를 이용한 계산 방법을 적용하여 시간 효율성을 높이고, 다양한 방향에서 촬영된 사진을 결합하여 지형 정보를 보다 자세히 표현할 수 있는 기술을 개발할 필요가 있다. 이러한 기술을 통해 더 현실적인 3D 지형 데이터를 생성할 수 있을 것으로 기대된다.

■ 참고 문헌

[1] 이규복. (2024). 생성형 AI를 활용한 터레인 텍스처(Terrain Texture) 적용 방안 연구. 한국게임학회 논문지, 24(3), 3-14.

VI. 부록

```
!apt-get update
!apt-get install -y gdal-bin python3-gdal python3-numpy
!pip install rasterio matplotlib

import cv2
import sys
import numpy as np
import torch
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from torchvision.transforms import Compose, Resize, ToTensor, Normalize
from PIL import Image
from matplotlib import pyplot as plt
from osgeo import gdal, osr
import rasterio

def load_midas_model():
    model_type = "DPT_Large"
    midas = torch.hub.load("intel-isl/MiDaS", model_type)
    midas.eval()

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    midas.to(device)

    transforms = torch.hub.load("intel-isl/MiDaS", "transforms")
    transform = transforms.dpt_transform if "DPT" in model_type else transforms.small_transform

    return midas, transform, device

def estimate_depth(image_path, model, transform, device):
    img = cv2.imread(image_path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    input_batch = transform(img).to(device)

    with torch.no_grad():
        prediction = model(input_batch)

        prediction = torch.nn.functional.interpolate(
            prediction.unsqueeze(1),
            size=img.shape[:2],
            mode="bicubic",
            align_corners=False,
        ).squeeze()

    depth_map = prediction.cpu().numpy()
    return depth_map

def normalize_depth(depth_map, max_val):
    normalized = (depth_map - np.min(depth_map)) / (np.max(depth_map) - np.min(depth_map))
    scaled = normalized * max_val
    return scaled

def visualize_depth(image_path, depth_map):
    original_image = cv2.imread(image_path)
    original_image = cv2.cvtColor(original_image, cv2.COLOR_BGR2RGB)

    plt.figure(figsize=(12, 6))

    plt.subplot(1, 2, 1)
    plt.title("Original Image")
    plt.imshow(original_image)
    plt.axis("off")

    plt.subplot(1, 2, 2)
    plt.title("Normalized Height Map")
    plt.imshow(depth_map, cmap="viridis")
    plt.colorbar(label="Height (0 to 100)")
    plt.axis("off")

    plt.show()
```

```
def convert_to_top_view(front_distance_map):
    H, W = front_distance_map.shape
    tmp_map =  np.full((H, W), W)
    for i in range(H):
        for j in range(W):
          for k in range(H - int(front_distance_map[i][j]), H):
            tmp_map[k][j] -= 1
    return np.array(tmp_map)

def visualize_3d(Z):
    x = np.arange(Z.shape[1])
    y = np.arange(Z.shape[0])
    X, Y = np.meshgrid(x, y)

    X -= X.min()
    Y -= Y.min()
    Z -= Z.min()

    fig = plt.figure(figsize=(10, 7))
    ax = fig.add_subplot(111, projection='3d')
    surface = ax.plot_surface(X, Y, Z, cmap='gray', edgecolor='none')

    ax.set_title('3D Height Map Visualization')
    ax.set_xlabel('X Axis')
    ax.set_ylabel('Y Axis')
    ax.set_zlabel('Height')

    plt.show()

def array_to_geotiff(array, output_path, transform=None, crs="EPSG:4326"):
    rows, cols = array.shape
    driver = gdal.GetDriverByName("GTiff")
    dataset = driver.Create(output_path, cols, rows, 1, gdal.GDT_Float32)

    if transform is None:
        transform = [0, 1, 0, 0, 0, -1]
    dataset.SetGeoTransform(transform)

    srs = osr.SpatialReference()
    srs.ImportFromEPSG(int(crs.split(":")[1]))
    dataset.SetProjection(srs.ExportToWkt())

    dataset.GetRasterBand(1).WriteArray(array)
    dataset.FlushCache()
    dataset = None
    print(f"GeoTIFF 저장 완료: {output_path}")

def display_geotiff(file_path):
    with rasterio.open(file_path) as dataset:
        data = dataset.read(1)
        transform = dataset.transform
        crs = dataset.crs

        print("파일 경로:", file_path)
        print("크기 (Height, Width):", dataset.height, dataset.width)
        print("좌표계 (CRS):", crs)
        print("GeoTransform:", transform)
        print("최소값:", data.min(), "최대값:", data.max())

        plt.figure(figsize=(8, 6))
        plt.imshow(data, cmap="terrain")
        plt.colorbar(label="Height")
        plt.title("Elevation Map")
        plt.xlabel("X Pixel")
        plt.ylabel("Y Pixel")
        plt.show()

def main(image_path):
    midas, transform, device = load_midas_model()
    depth_map = estimate_depth(image_path, midas, transform, device)
    depth_map[depth_map < 0] = 0
    height_map = normalize_depth(depth_map, 100)
    height_map = np.max(height_map) - height_map
    visualize_depth(image_path, height_map)
```

```
top_view_depth_map = normalize_depth(height_map, depth_map.shape[0])
    top_view_height_map = normalize_depth(convert_to_top_view(top_view_depth_map), 100)
    visualize_depth(image_path, top_view_height_map)
    visualize_3d(top_view_height_map)
    array_to_geotiff(top_view_height_map, "/content/elevation_map.tif")
    display_geotiff("/content/elevation_map.tif")


if __name__ == "__main__":
    test_image_path = 'image.jpg'
    main(test_image_path)
```